

INF01085 - Relatório do Trabalho Prático 1

- Nomes: Eduardo Fonseca da Silva, Estevan Zanetti Küster

1. Visão geral

Este trabalho implementa um serviço cliente-servidor sobre o protocolo UDP. Como o UDP não oferece confiança, a solução foi projetada com um protocolo de aplicação para garantir um comportamento confiável. A confiabilidade foi implementada com um mecanismo Stop-and-Wait com controle de sequência, garantindo tolerância a perdas, duplicatas e reordenação de pacotes.

2. Implementação de cada subserviço

2.1 Subserviço de descoberta

Cliente (`client/discovery.py`): O cliente utiliza um socket configurado para permitir envio em broadcast (`SO_BROADCAST`). Ele envia a mensagem `DESCOBERTA` para a porta informada e entra em estado de espera bloqueante (com timeout).

Servidor (`server/processing.py`): Ao receber o datagrama de `DESCOBERTA`, o servidor extrai o IP e a porta de origem via `recvfrom` e responde diretamente com a mensagem `IP_SERVIDOR_OK`. Neste momento, o servidor registra o novo cliente na estrutura `state.estado_clientes`.

2.2 Subserviço de processamento

Cliente (`client/processing.py`): Utiliza o protocolo stop-and-wait. A função empacota o envio no formato `id_req, valor`, envia e aguarda o ACK correspondente antes de liberar o próximo envio.

Servidor (`server/processing.py`): O servidor valida o `id_req` recebido comparando ele com o próximo ID esperado daquele cliente específico:

- **Comportamento ideal:** Se o ID é o esperado, processa a soma e responde com um ACK atualizado.
- **Duplicatas:** Se `id < esperado`, é um pacote duplicado (provavelmente o ACK anterior se perdeu). Nesse caso, reenvia-se o ACK com o último estado válido para destravar o cliente.
- **Reordenação:** Se `id > esperado`, o pacote chegou fora de ordem. O servidor descarta o processamento e envia um ACK com o último estado conhecido.
- **(RESET):** Caso o servidor seja reiniciado, ele perde todas as informações dos clientes porque ele não guarda as informações em disco (e nem deve guardar). Por isso, foi desenvolvido um tratamento para este caso. Se um cliente desconhecido tentar enviar um pacote com `id > 1`, o servidor envia uma mensagem de `RESET`, forçando o cliente a reiniciar sua contagem (`id_req = 1`). Então o cliente manda a mensagem novamente com `id_req = 1` e a soma acumulada também será reiniciada a partir da resposta do servidor.

2.3 Subserviço de interface

Cliente (`client/interface.py`): Utiliza threads separadas para entrada de dados (teclado) e lógica de rede/saída, garantindo que o cliente não bloqueie a leitura de novos inputs do usuário enquanto aguarda um ACK. Os logs exibem timestamps e campos de protocolo.

Servidor (`server/interface.py`): Mantém a exibição em tempo real do estado de processamento (`num_reqs` e `total_sum`) e de anomalias na rede (duplicatas, fora de ordem).

3. Áreas com necessidade de sincronização de dados

3.1 Sincronização local (concorrência em memória)

Cliente (produtor-consumidor via `Queue`): O cliente possui uma thread bloqueada aguardando a entrada do usuário (`stdin`) e outra dedicada ao laço principal de rede. A área crítica aqui é a passagem da mensagem digitada para a thread de envio. A sincronização de dados foi resolvida utilizando a estrutura `Queue` nativa da linguagem. Como a `Queue` já implementa locks internos, eliminou-se a necessidade de gerenciar mutexes manualmente.

Servidor (isolamento por single-thread): A decisão foi implementar o servidor com um loop de eventos single-thread.

3.2 Sincronização distribuída (consistência inter-processos)

Sincronização de estado (cliente <-> servidor): Como os processos não compartilham memória e o UDP não garante ordem, foi necessário sincronizar o estado lógico da aplicação. A área de código que garante isso é a validação do `id_requisicao` no cliente com o `id_esperado` no servidor explicada anteriormente. Essa sincronização lógica garante que o servidor só adicione o valor ao `acumulador_global` se houver um consenso de que aquele pacote é, de fato, o próximo da fila.

4. Principais estruturas e funções implementadas

Estruturas de estado (`server/state.py`)

- **ServerState** : Estrutura global que centraliza a fonte de verdade do servidor. Armazena as métricas globais (`num_requisicoes_total` e `acumulador_global`) e um dicionário `estado_clientes` que mapeia cada cliente ativo (endereço IP e porta) para o seu respectivo estado.
- **ClientState** : Estrutura instanciada para cada cliente único descoberto. Armazena o ID da última requisição recebida com sucesso (`last_req`), além da foto do estado do servidor naquele momento exato (`last_num_reqs` , `last_total_sum`).

Funções principais no cliente

- **descobrir_servidor** : Gerencia a criação do socket de broadcast, envio da `DESCOBERTA` e extração do IP/porta da resposta.
- **enviar_valor_stop_and_wait** : Encapsula a lógica de formatação da mensagem, envio `sendto` e o loop de recebimento bloqueante com `settimeout` para gerenciar retransmissões.

- **executar_loop_principal** : Consome a fila de inputs do usuário e coordena as chamadas sequenciais para a função de envio, controlando o incremento do `id_req` .

Funções principais no servidor

- **executar_loop_servidor** : O laço principal (`while True`) que atua como listener bloqueante em `recvfrom` , roteando mensagens de descoberta ou processamento.
- **handle_descoberta** : Processa novos clientes, os registra na tabela de estado interno e despacha o `IP_SERVIDOR_OK` .
- **handle_processamento** : Executa a validação crítica de protocolo (checagem de sequência, processamento da soma, detecção de falhas de ordem) e empacota/envia os ACKs e RESETs.

5. Uso das primitivas de comunicação inter-processos

As primitivas de IPC basearam-se puramente na API de Sockets Berkeley:

- **Criação de sockets**: Uso de `AF_INET` para endereçamento IPv4 e `SOCK_DGRAM` indicando o uso do protocolo de transporte UDP.
- **Opções de socket**: Uso de `setsockopt` com a flag `SO_BROADCAST` no cliente para habilitar o envio da mensagem de descoberta para a máscara `255.255.255.255` .
- **Troca de mensagens**: Uso de `sendto` e `recvfrom` .
- **Temporização (timeout)**: Como o UDP não reporta perdas, a primitiva `settimeout` foi aplicada ao socket do cliente. Esta primitiva levanta uma exceção nativa no nível do sistema operacional caso a chamada do sistema (`syscall`) de recebimento expire, servindo de gatilho para a retransmissão de pacotes.

6. Problemas encontrados e soluções

6.1 Acúmulo de ACKs antigos (buffer drain)

Problema: Foi notado que, em redes com alta latência, quando um cliente retransmitia um pacote por timeout e, logo em seguida, dois ACKs chegavam, o segundo ACK (que estava atrasado) ficava preso no buffer do sistema operacional. Na requisição seguinte, a primitiva `recvfrom` lia imediatamente este ACK antigo, causando dessincronização e novas retransmissões indevidas.

Solução: Implementou-se um mecanismo de drain (drenagem) dentro de `enviar_valor_stop_and_wait` . O loop interno lê a resposta e extrai o ID que o ACK confirma. Se não for igual ao ID atual do envio, o pacote é ativamente descartado e o socket continua escutando. A retransmissão só ocorre se o timeout do socket estourar.

6.2 Perda de contexto por queda do servidor

Problema: O modelo de falha clássico onde o servidor sofre crash e volta ao ar. A especificação não detalhava o procedimento de recuperação. O cliente, sem saber da queda, enviava seu próximo valor com `id_req = 5` , mas o servidor recém-reiniciado esperava `id_req = 1` , causando um deadlock pois o servidor enxergava como pacote fora de ordem e o cliente ficava em loop de retransmissão.

Solução: Adição da mensagem de controle `RESET`. Se o servidor identifica uma tupla de cliente desconhecida na memória cujo `id_req` seja maior que 1, ele infere que houve uma perda de memória e responde com `RESET`. O cliente processa esse comando e reinicia sua sequência.

6.3 Resolução de ambiguidade de clientes

Problema: No início, clientes eram identificados apenas pelo endereço IP. No entanto, se o avaliador rodasse múltiplas instâncias do cliente na mesma máquina física (mesmo localhost ou mesma rede NAT), o servidor não conseguiria distinguir as requisições.

Solução: O dicionário `estado_clientes` foi refatorado para utilizar a tupla completa (`IP`, `Porta_Origem`) gerada pelo sistema operacional no `recvfrom` como chave única de identificação. Para manter tudo na especificação de logs exigida, a camada de interface formata as saídas imprimindo apenas o IP.

Tutorial de uso

Pré-requisitos

- Python 3 instalado.
- Cliente e servidor na mesma rede local.

1. Iniciar o servidor

```
python server\main.py <porta para servidor>
```

2. Iniciar o cliente

```
python client\main.py <porta para o servidor>
```

3. Enviar valores para soma

Com o cliente em execução, digite um inteiro por linha e pressione Enter.

4. Encerrar execução

- Para parar cliente/servidor: `Ctrl + C` em cada terminal.
- Também é possível encerrar o cliente enviando EOF no terminal (tipicamente `Ctrl+Z` no Windows ou `Ctrl+D` no Linux).